



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización de Arquitectura de Computadoras

Reporte - Diseño de máquinas de estado

NOMBRE DE LOS INTEGRANTES:

Omar Alfonso Zendejas Labias Grupo de Teoría: 04

Emiliano García Olvera Grupo de Teoría: 02

NOMBRE DEL PROFESOR: ING. KAREN PÉREZ

GRUPO: 06 EQUIPO: 9

SEMESTRE: 2027-1.

FECHA DE ENTREGA: 15 de septiembre de 2026

Objetivo

Se busca analizar, diseñar e implementar una máquina de estados finitos a partir de una carta ASM proporcionada, identificando el funcionamiento de los *flip-flops* tipo D como elementos de memoria. A partir del llenado de la tabla de transiciones y de la reducción mediante mapas de Karnaugh se obtienen las funciones de estado siguiente y de salida, las cuales posteriormente se implementan en VHDL y se verifican sobre la tarjeta de desarrollo FPGA DE10-Lite.

Introducción

Una carta ASM (*Algorithmic State Machine*) es una representación gráfica del comportamiento de un circuito secuencial, equivalente a un diagrama de estados, en la que cada estado se dibuja como una caja rectangular y cada condición de transición como una caja de decisión. A diferencia del diagrama de estados clásico, la carta ASM se asemeja a un diagrama de flujo y resulta más directa para derivar, a partir de ella, la tabla de transiciones del sistema.

En una implementación síncrona, el estado de la máquina se almacena en un conjunto de *flip-flops* tipo D, que actúan como elementos de memoria: en cada flanco de *clock* capturan el valor calculado como estado siguiente y lo mantienen hasta el siguiente flanco. Las entradas *D* de estos *flip-flops*, así como las señales de salida de la máquina, se obtienen como funciones booleanas del estado actual y de las entradas del sistema; dichas funciones se simplifican mediante mapas de Karnaugh a partir de la tabla de transiciones, siguiendo el procedimiento estándar de diseño de circuitos secuenciales síncronos descrito, entre otros, por Savage Carmona *et al.* [1].

Desarrollo

Parte I. Análisis y diseño de la máquina de estados

La carta ASM proporcionada para esta práctica se muestra en la figura 1.

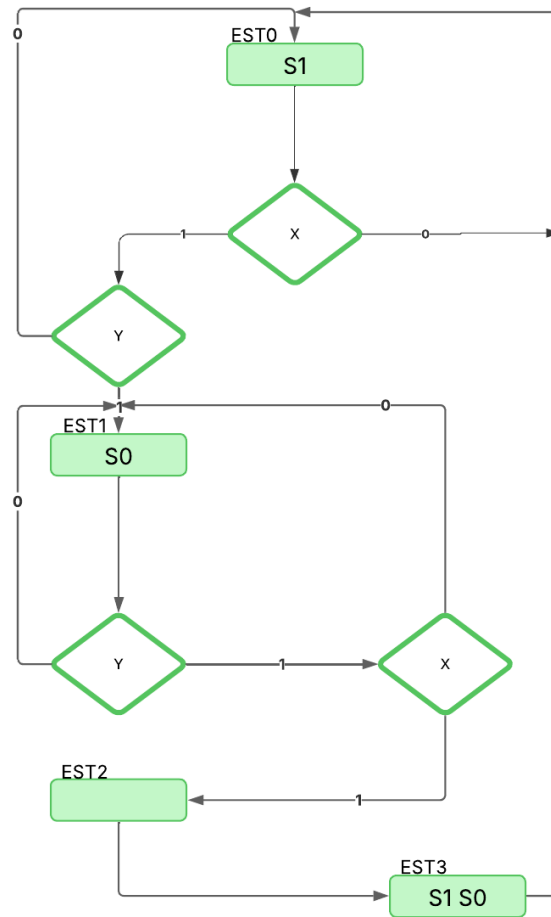


Figura 1: Carta ASM del sistema, proporcionada en el manual de la práctica.

3.1.1. Identificación de estados, entradas y salidas

La carta ASM define cuatro estados: EST0, EST1, EST2 y EST3. La máquina tiene dos entradas, X y Y , y dos señales de salida, S_1 y S_0 , cuyo valor depende únicamente del estado actual (es decir, se trata de una máquina de tipo Moore): EST0 activa S_1 ($S_1 S_0 = 10$), EST1 activa S_0 (01), EST2 no activa ninguna salida (00) y EST3 activa ambas (11).

3.1.2. Codificación de los estados

Al ser cuatro estados se requieren 2 bits, almacenados en dos flip-flops tipo D con salidas $Q_1 Q_0$. Se propuso la codificación secuencial: EST0= 00, EST1= 01, EST2= 10, EST3= 11.

3.1.3. Tabla de transiciones

Siguiendo las rutas de decisión de la carta ASM (figura 1) se obtiene la siguiente tabla de transiciones, en la que un asterisco indica que la salida de ese estado no depende de dicha entrada:

Estado actual		Entradas		Estado siguiente		Salidas	
Edo.	Q_1Q_0	X	Y	Edo.	Q_1Q_0	S_1	S_0
EST0	00	0	*	EST0	00	1	0
EST0	00	1	0	EST0	00	1	0
EST0	00	1	1	EST1	01	1	0
EST1	01	*	0	EST1	01	0	1
EST1	01	0	1	EST1	01	0	1
EST1	01	1	1	EST2	10	0	1
EST2	10	*	*	EST3	11	0	0
EST3	11	*	*	EST0	00	1	1

Tabla 1: Tabla de transiciones obtenida a partir de la carta ASM.

3.1.4. Mapas de Karnaugh

A partir de la tabla 1 se construyeron los mapas de Karnaugh de las entradas D_1 , D_0 de los flip-flops y de las salidas S_1 , S_0 , usando como ejes Q_1Q_0 y XY :

$Q_1Q_0 \backslash XY$	00	01	11	10	$Q_1Q_0 \backslash XY$	00	01	11	10
00	0	0	0	0	00	0	0	1	0
01	0	0	1	0	01	1	1	0	1
11	0	0	0	0	11	0	0	0	0
10	1	1	1	1	10	1	1	1	1
D_1					D_0				
$Q_1Q_0 \backslash XY$	00	01	11	10	$Q_1Q_0 \backslash XY$	00	01	11	10
00	1	1	1	1	00	0	0	0	0
01	0	0	0	0	01	1	1	1	1
11	1	1	1	1	11	1	1	1	1
10	0	0	0	0	10	0	0	0	0
S_1					S_0				

Figura 2: Mapas de Karnaugh de D_1 , D_0 , S_1 y S_0 . En D_1 , el grupo azul (renglón $Q_1Q_0 = 10$ completo) da el término $Q_1\overline{Q_0}$ y la celda naranja aislada da $\overline{Q_1}Q_0XY$. En D_0 se agrupan el renglón $Q_1Q_0 = 10$ ($Q_1\overline{Q_0}$), la celda $Q_1Q_0 = 00, XY = 11$ ($\overline{Q_0}XY$), y dentro del renglón $Q_1Q_0 = 01$ las columnas $XY = 00, 01$ ($\overline{Q_1}Q_0\overline{X}$) y $XY = 00, 10$ ($\overline{Q_1}Q_0\overline{Y}$). En S_1 los dos renglones llenos dan $\overline{Q_1}Q_0$ y Q_1Q_0 . En S_0 los renglones $Q_1Q_0 = 01$ y 11 forman un solo grupo de ocho celdas: Q_0 .

3.1.5. Funciones booleanas simplificadas

De los mapas de Karnaugh de la figura 2 se obtienen las siguientes funciones simplificadas:

$$D_1 = Q_1 \overline{Q_0} + \overline{Q_1} Q_0 XY \quad (1)$$

$$D_0 = Q_1 \overline{Q_0} + \overline{Q_0} XY + \overline{Q_1} Q_0 \overline{X} + \overline{Q_1} Q_0 \overline{Y} \quad (2)$$

$$S_1 = \overline{Q_1} \overline{Q_0} + Q_1 Q_0 \quad (3)$$

$$S_0 = Q_0 \quad (4)$$

3.1.6. Circuito resultante

El circuito de la figura 3 (sección Parte II), obtenido al compilar el proyecto en Quartus Prime, implementa directamente las ecuaciones 1-4 mediante compuertas AND, OR y NOT que alimentan las entradas D de dos flip-flops tipo D.

Parte II. Implementación en Quartus y FPGA DE10-Lite

Como base para generar la señal de reloj lenta necesaria para observar las transiciones de la máquina de estados en la FPGA, la profesora proporcionó el módulo divisor de frecuencia `div_frec.vhd`, mostrado en la figura 1.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity div_frec is
7     Port (reloj : in std_logic;
8           div_clk : out std_logic);
9 end div_frec;
10
11 architecture Behavioral of div_frec is
12 begin
13     process (reloj)
14         variable cuenta : std_logic_vector (27 downto 0) := X"00000000";
15         begin
16             if rising_edge (reloj) then
17
18                 --if cuenta 0 "40000000" then
19                 if cuenta = X"2FAF080" then
20                     cuenta := X"00000000";
21                 else
22                     cuenta := cuenta+1;
23                 end if;
24             end if;
25             div_clk <= cuenta(25);
26         end process;
27     end Behavioral;

```

Figura 1: Módulo `div_frec.vhd`, proporcionado por la profesora como base del proyecto.

Este módulo es el punto de partida entregado por la profesora. La máquina de estados en sí se capturó como diagrama esquemático (`practica2_ff.bdf`), no como código VHDL propio; falta exportar/escribir ese `.bdf` o su equivalente en código/ para dejar evidencia completa del proyecto (ver nota en “Implementación”, más abajo).

3.2.1. Creación del proyecto en Quartus

Se creó el proyecto `practica2_ff` en Quartus Prime Lite Edition 18.1 para el dispositivo MAX 10 - 10M50DAF484C7G de la tarjeta DE10-Lite, tal como lo especifica el manual (figura 4).

3.2.2. Diagrama lógico e integración con `div_frec.vhd`

La figura 3 muestra el esquemático compilado del proyecto: el módulo `div_frec` (parte inferior) recibe el reloj de la tarjeta y entrega la señal de reloj lenta que alimenta a los dos flip-flops tipo D de la máquina de estados; las entradas `PIN_C10/PIN_C11 (X, Y)` y la red de compuertas AND/OR/NOT

que implementa las ecuaciones 1-4 producen las salidas S1 y S0.

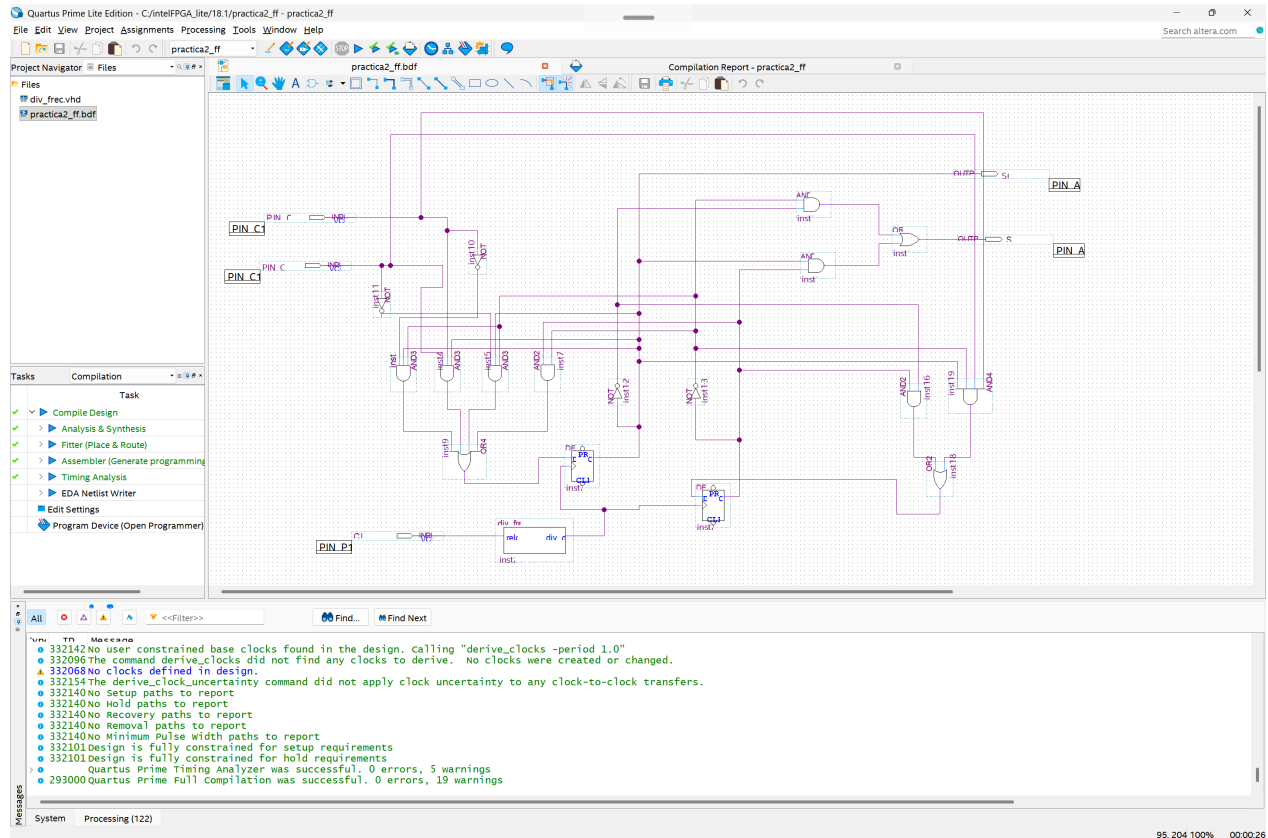


Figura 3: Esquemático compilado (practica2_ff.bdf) en Quartus Prime, integrando div_freq con la lógica de la máquina de estados.

3.2.3. Implementación

El manual pide implementar la máquina “mediante VHDL” (Parte II, paso 3); lo que se compiló fue un diagrama esquemático (.bdf) con compuertas individuales, no un módulo VHDL de la máquina de estados. Confirmar con la profesora si el esquemático es aceptado como equivalente o si además hay que escribir el código VHDL correspondiente a las ecuaciones 1-4 y agregarlo a código/.

3.2.4. Asignación de pines

La asignación de entradas y salidas a los recursos físicos de la DE10-Lite, tomada del *Pin Planner* de Quartus, se muestra en la tabla 2 y en la figura 4.

Señal	Dirección	Pin	Descripción
CLK	Entrada	PIN_P11	Reloj de la tarjeta (50 MHz)
PIN_C10	Entrada	PIN_C10	Entrada X
PIN_C11	Entrada	PIN_C11	Entrada Y
S0	Salida	PIN_A8	Salida S_0
S1	Salida	PIN_A9	Salida S_1

Tabla 2: Asignación de pines del proyecto practica2_ff.

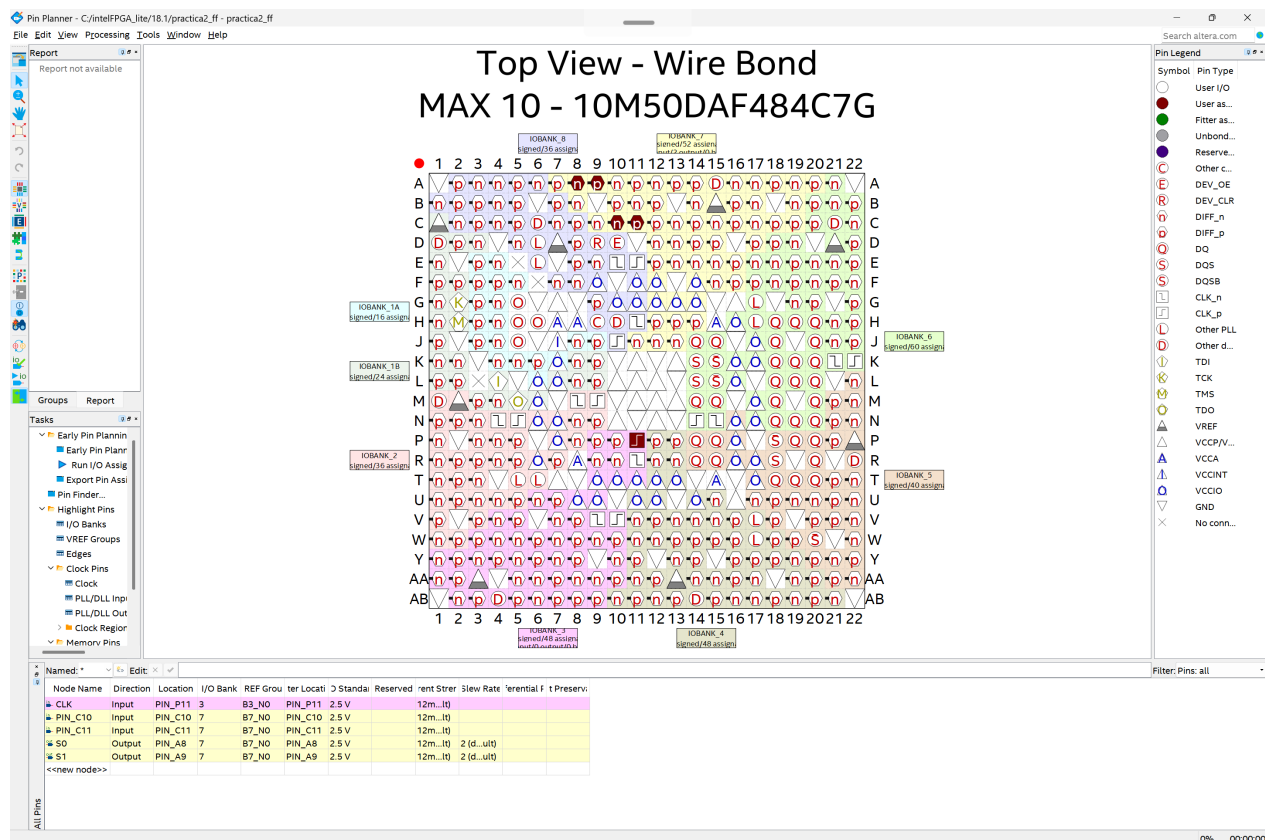


Figura 4: Pin Planner de Quartus Prime mostrando el dispositivo (MAX 10 – 10M50DAF484C7G) y la tabla de asignación de pines.

Documentar a qué recurso físico de la tarjeta corresponde cada pin (por ejemplo, PIN_C10/PIN_C11 a los interruptores SW usados como X/Y , y PIN_A8/PIN_A9 a los LEDs usados como S_0/S_1), según la hoja de pines oficial de la DE10-Lite.

3.2.5. Compilación y programación de la FPGA

El proyecto compiló sin errores: el reporte de compilación de la figura 3 indica “*Quartus Prime Full Compilation was successful. 0 errors, 19 warnings*”. La FPGA se programó y se comprobó su funcionamiento físicamente, como se muestra en la figura 5.

Revisar y, si aplica, describir en el reporte a qué se deben las 19 advertencias de compilación (*warnings*); usualmente no impiden el funcionamiento pero conviene documentarlas.

3.2.6. Comprobación experimental

Se tomaron fotografías de la tarjeta DE10-Lite programada con distintas combinaciones de las entradas X, Y (figura 5).



$XY = 00$



$XY = 01$



$XY = 11$

Figura 5: Tarjeta DE10-Lite programada, con los interruptores en las combinaciones $XY = 00, 01$ y 11 .

Las fotografías confirman que la tarjeta quedó programada y encendida (LEDs y *power good* activos), pero por el ángulo/iluminación no se alcanza a leer con certeza qué LEDs específicos codifican S_1S_0 en cada captura. Completar: (1) describir qué LEDs/recursos muestran el estado y la salida en cada foto, (2) agregar la combinación $XY = 10$ que falta, y (3) para cada captura, indicar en qué estado (EST0-EST3) se encontraba la máquina y compararlo contra la tabla 1.

Análisis de resultados

De acuerdo con las secciones anteriores, el diseño de la Parte I (tabla de transiciones, mapas de Karnaugh y ecuaciones 1-4) se implementó y compiló sin errores en Quartus, y la tarjeta fue programada y responde a los interruptores de entrada.

Falta la comparación explícita, transición por transición, entre el comportamiento observado en la FPGA (figura 5, y las combinaciones que falten por capturar) y lo previsto en la tabla 1 de la Parte I, señalando si coinciden o si se encontró alguna discrepancia.

Conclusiones individuales

Omar Alfonso Zendejas Labias

Redactar la conclusión individual una vez concluidas ambas partes de la práctica.

Emiliano García Olvera

Redactar la conclusión individual una vez concluidas ambas partes de la práctica.

Referencias

- [1] J. Savage Carmona, G. Vázquez y N. E. Chávez Rodríguez, *Diseño de microprocesadores*. Ciudad de México, México: Facultad de Ingeniería, Universidad Nacional Autónoma de México, 2017. [En línea]. Disponible: <http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf>